Lecture 19 — Disjoint Set Union and Minimum Spanning Trees

> Year Curriculum · Prerequisites: Week 4 (sorting, exchange arguments), Week 13 (priority queues), Week 16 (graphs, components) · Estimated time: 2.5 hours

1. Overview

In Week 16 we computed connected components with one BFS or DFS over a fixed graph. Many problems instead *add edges over time* and ask about connectivity after each addition. Recomputing components each time costs O(n + m) per query, far too slow for 10^5 updates.

The **Disjoint Set Union** (DSU, also called *union-find*) maintains a partition of {1, …, n} into groups under two operations — *find the group of x* and *merge two groups* — in amortised time so close to constant that we treat it as O(1). It is one of the shortest and most frequently used data structures in contests.

The headline application is the **minimum spanning tree** (MST): the cheapest set of edges connecting all vertices. We will prove the *cut property* that justifies the greedy MST algorithms, implement **Kruskal's** algorithm (sort + DSU) and **Prim's** algorithm (heap), and finish with the powerful *offline* pattern: sort events, sweep, and merge.

2. Disjoint Set Union

2.1 Representation

Each group is stored as a rooted tree using a parent array `p`. A root has `p[r] = r` and acts as the group's representative.

- `find(x)`: follow parent pointers from x until reaching the root.
- `unite(a, b)`: find both roots; if different, make one the parent of the other.
- `a` and `b` are in the same group iff `find(a) == find(b)`.

Initially every element is its own group: `p[i] = i`.

Without care the trees can degenerate into long paths (unite(1,2), unite(2,3), …, always hanging the old root under the new element), making `find` O(n). Two simple optimisations fix this.

2.2 Union by size

Store `sz[r]` = number of elements in r's tree (meaningful only for roots). When uniting, attach the root of the **smaller** tree below the root of the larger one.

Claim: every tree of size s has height at most $\log_2 s$. An element's depth increases by one only when its tree is attached below another tree at least as large, so the size of the tree containing it at least doubles. This can happen at most $\log_2 n$ times. Hence `find` is O(log n) even without any other trick.

A pleasant side effect is that `sz[find(x)]` gives the size of x's component for free. (Union by *rank*, which tracks an upper bound on height instead of size, gives the same guarantee; size is more useful in practice.)

2.3 Path compression

During `find`, once we know the root, we can make every node on the path point closer to (or directly at) the root, so the next query on them is fast. The fully compressing recursive version is

```
int find(int x) { return p[x] == x ? x : p[x] = find(p[x]); }
```

and the iterative *path halving* variant, which needs no recursion, is

```
int find(int x) {
    while (p[x] != x) { p[x] = p[p[x]]; x = p[x]; }
    return x;
}
```

Path halving makes every node on the path point to its grandparent. It achieves the same asymptotic bound and avoids deep recursion on worst-case inputs.

2.4 Complexity

With both union by size and path compression, any sequence of m operations on n elements takes $O(m \cdot \alpha(n))$ time, where α is the inverse Ackermann function (Tarjan, 1975). $\alpha(n) \le 4$ for every n smaller than the number of atoms in the universe, so in practice each operation is constant time. With only one of the two optimisations you get O(log n) per operation — usually also fast enough, but there is no reason not to use both.

2.5 What DSU supports, and what it doesn't

DSU handles **merging** and **same-group queries**, plus any per-group aggregate that can be merged in O(1): size, sum, minimum, number of edges (to detect whether a component is a tree), and so on — just combine the aggregates of the two roots in `unite`.

It also detects cycles: when adding an undirected edge (a, b), if `find(a) == find(b)` the edge closes a cycle. That is exactly the test inside Kruskal.

DSU **cannot split** groups or delete edges. The standard workaround, when all queries are known in advance, is to **reverse time**: process the operations backwards so that deletions become additions. CSES 1677 *Network Breakdown* and USACO *Closing the Farm* are classic examples. (Fully dynamic connectivity is a World Finals-level topic, see the offline techniques lecture in that track.)

2.6 Worked example — CSES 1676 *Road Construction*

n cities and m roads added one by one; after each addition print the number of components and the size of the largest component.

Start with `comps = n` and `best = 1`. For each road, if `unite(a, b)` merges two groups, decrement `comps` and update `best` with the new root's size. The largest component never shrinks, so a running maximum suffices. Total O((n + m)·α(n)). Sample: after roads 1–2, 1–3, 4–5 with n = 5, the outputs are `4 2`, `3 3`, `2 3`.

3. Minimum spanning trees

3.1 Definitions

Let G be a connected, undirected, weighted graph. A **spanning tree** is a subset of n − 1 edges that connects all vertices (and therefore has no cycle). A **minimum spanning tree** minimises the total weight. If G is disconnected, no spanning tree exists; the natural generalisation is a minimum spanning *forest*.

CSES 1675 *Road Reparation*: n cities, m possible roads with repair costs; find the minimum total cost to connect all cities, or print IMPOSSIBLE.

3.2 The cut property

A **cut** (S, V\S) partitions the vertices into two non-empty sets; an edge *crosses* the cut if it has one endpoint on each side.

> **Cut property.** For any cut, any minimum-weight edge crossing it belongs to some MST.

Proof. Let e = (u, v) be a minimum crossing edge and T an MST not containing e. Adding e to T creates a cycle; this cycle goes from u to v, so besides e it crosses the cut at least once more, via some edge f. Since e is minimum among crossing edges, w(e) ≤ w(f). Then T' = T − f + e is still a spanning tree (removing f breaks the cycle we created, not connectivity) and w(T') ≤ w(T), so T' is also an MST and contains e. ∎

The companion *cycle property* says the heaviest edge on any cycle (if unique) is in no MST. Both are exchange arguments in the sense of Week 4: MST is a problem where greedy provably works.

3.3 Kruskal's algorithm

Sort the edges by weight. Scan them in increasing order and take an edge if its endpoints are currently in different components; DSU answers that question and performs the merge.

Correctness. When Kruskal takes edge (a, b), consider the cut (component of a, everything else). No edge crossing this cut has been seen yet (it would have been taken, merging the components), so (a, b) is a minimum crossing edge; by the cut property it is safe. Skipped edges close cycles and are never needed.

Complexity. O(m log m) for sorting plus O(m·α(n)) for the DSU.

Trace on the CSES sample (edges 1–2:3, 2–3:5, 2–4:2, 3–4:8, 5–1:7, 5–4:4):

edge	weight	action	components after
2–4	2	take	{1} {2,4} {3} {5}
1–2	3	take	{1,2,4} {3} {5}
5–4	4	take	{1,2,4,5} {3}
2–3	5	take	{1,2,3,4,5}
1–5	7	skip (cycle)	
3–4	8	skip (cycle)	

Total 14 with 4 = n − 1 edges. If fewer than n − 1 edges are taken at the end, the graph is disconnected.

Storing edges as `array<int,3>{w, a, b}` lets plain `sort` order them by weight with no custom comparator.

3.4 Prim's algorithm

Prim grows a single tree from an arbitrary start vertex. At each step it adds the cheapest edge connecting the tree to a vertex outside it — the cut property with S = current tree. With a priority queue of (edge weight, vertex) and lazy deletion, it looks almost exactly like Dijkstra:

```
pq.push({0, 1});
while (!pq.empty()) {
    auto [w, u] = pq.top(); pq.pop();
    if (inTree[u]) continue;
    inTree[u] = true; total += w;
    for (auto [v, c] : adj[u]) if (!inTree[v]) pq.push({c, v});
}
```

The crucial difference: the priority is the **weight of the single connecting edge** `c`, not a path length `dist[u] + c`. Using the Dijkstra key computes a shortest-path tree, which is generally *not* an MST.

Complexity O(m log m). For dense graphs (m ≈ n^2, e.g. complete graphs given by coordinates), the O(n^2) array version of Prim — repeatedly scan all vertices for the minimum key — beats any O(m log m) method and avoids storing n^2 edges.

3.5 Choosing between them

Kruskal is the contest default: short, works on an edge list, handles forests naturally, and its sorted sweep generalises to offline problems. Prim is preferable for dense graphs given implicitly, or when you already have adjacency lists and want to avoid a DSU.

3.6 Useful MST facts

- If all edge weights are distinct, the MST is unique.
- A **maximum** spanning tree: sort in decreasing order.
- **Minimax paths:** in an MST, the path between u and v minimises the maximum edge weight among all u–v paths in the graph. Questions like "the smallest possible largest edge on a route" are MST questions.
- **Forced edges:** unite them before the sweep (as if their weight were $-\infty$).
- Negative weights need no special handling; only the order matters.
- Kruskal processes edges in increasing order, so after processing all edges of weight ≤ q, the DSU components are exactly the components of the subgraph of edges ≤ q. That is the basis of the offline technique below.

4. The offline sweep with DSU

CF 1213G *Path Queries*. A weighted tree with n vertices and m queries; for each q, count pairs u < v such that the maximum edge weight on the u–v path is at most q.

The pairs counted for q are exactly the pairs in the same component of the subgraph made of edges with weight ≤ q. So: sort edges by weight, sort the queries by q (remembering their original positions), and sweep. When an edge merges components of sizes s_1 and s_2, exactly

$s_1 \cdot s_2$ new pairs become connected. The answer to a query is the running total after adding all edges with weight ≤ q. Complexity O(n log n + m log m).

This pattern — **sort queries and updates by a parameter, sweep, maintain components with DSU** — appears constantly: "can I go from u to v using only edges of weight ≤ L?", "earliest time when everything is connected", "number of components after each deletion" (reverse time). Whenever queries are monotone in a threshold and the threshold only adds edges, think *offline DSU*.

5. Reference implementations

All compile with `g++ -std=c++17 -O2 -Wall` and produce the expected sample outputs.

5.1 DSU — CSES 1676 *Road Construction*

```cpp
#include <bits/stdc++.h>
using namespace std;
struct DSU {
    vector<int> p, sz;
    DSU(int n) : p(n), sz(n, 1) { iota(p.begin(), p.end(), 0); }
    int find(int x) {
        while (p[x] != x) {
            p[x] = p[p[x]];               // path halving
            x = p[x];
        }
        return x;
    }
    bool unite(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return false;
        if (sz[a] < sz[b]) swap(a, b);     // union by size: attach smaller under larger
        p[b] = a;
        sz[a] += sz[b];
        return true;
    }
};
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    DSU d(n + 1);
    int comps = n, best = 1;
    while (m--) {
        int a, b;
        cin >> a >> b;
        if (d.unite(a, b)) {
            comps--;
            best = max(best, d.sz[d.find(a)]);
        }
```

```
        cout << comps << " " << best << "\n";
    }
}
```

5.2 Kruskal — CSES 1675 *Road Reparation*

```
#include <bits/stdc++.h>
using namespace std;
struct DSU {
    vector<int> p, sz;
    DSU(int n) : p(n), sz(n, 1) { iota(p.begin(), p.end(), 0); }
    int find(int x) {
        while (p[x] != x) {
            p[x] = p[p[x]];
            x = p[x];
        }
        return x;
    }
    bool unite(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return false;
        if (sz[a] < sz[b]) swap(a, b);
        p[b] = a;
        sz[a] += sz[b];
        return true;
    }
};
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<array<int, 3>> edges(m);         // (weight, a, b): sorts by weight first
    for (auto &[c, a, b] : edges) cin >> a >> b >> c;
    sort(edges.begin(), edges.end());
    DSU d(n + 1);
    long long total = 0;
    int used = 0;
    for (auto [c, a, b] : edges)
        if (d.unite(a, b)) {                // skip edges that would close a cycle
            total += c;
            used++;
        }
    if (used == n - 1) cout << total << "\n";
    else cout << "IMPOSSIBLE\n";
}
```

5.3 Prim — CSES 1675 *Road Reparation*

```
#include <bits/stdc++.h>
using namespace std;
```

```cpp
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<vector<pair<int, int>>> adj(n + 1);   // (neighbour, weight)
    for (int i = 0; i < m; i++) {
        int a, b, c;
        cin >> a >> b >> c;
        adj[a].push_back({b, c});
        adj[b].push_back({a, c});
    }
    vector<bool> inTree(n + 1, false);
    priority_queue<pair<int, int>, vector<pair<int, int>>, greater<>> pq; // (weight,
vertex)
    pq.push({0, 1});
    long long total = 0;
    int added = 0;
    while (!pq.empty()) {
        auto [w, u] = pq.top(); pq.pop();
        if (inTree[u]) continue;            // stale: u already joined by a cheaper edge
        inTree[u] = true;
        total += w;
        added++;
        for (auto [v, c] : adj[u])
            if (!inTree[v]) pq.push({c, v});
    }
    if (added == n) cout << total << "\n";
    else cout << "IMPOSSIBLE\n";
}
```

5.4 Offline DSU — CF 1213G *Path Queries*

```cpp
#include <bits/stdc++.h>
using namespace std;
struct DSU {
    vector<int> p, sz;
    DSU(int n) : p(n), sz(n, 1) { iota(p.begin(), p.end(), 0); }
    int find(int x) {
        while (p[x] != x) {
            p[x] = p[p[x]];
            x = p[x];
        }
        return x;
    }
    long long unite(int a, int b) {        // returns number of new connected pairs
        a = find(a); b = find(b);
        if (a == b) return 0;
        if (sz[a] < sz[b]) swap(a, b);
        long long added = 1LL * sz[a] * sz[b];
        p[b] = a;
        sz[a] += sz[b];
        return added;
```

```
    }
};
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<array<int, 3>> edges(n - 1);
    for (auto &[w, u, v] : edges) cin >> u >> v >> w;
    sort(edges.begin(), edges.end());
    vector<pair<int, int>> qs(m);
    for (int i = 0; i < m; i++) { cin >> qs[i].first; qs[i].second = i; }
    sort(qs.begin(), qs.end());
    DSU d(n + 1);
    vector<long long> ans(m);
    long long pairs = 0;
    size_t e = 0;
    for (auto [q, idx] : qs) {
        while (e < edges.size() && edges[e][0] <= q) {
            pairs += d.unite(edges[e][1], edges[e][2]);
            e++;
        }
        ans[idx] = pairs;
    }
    for (int i = 0; i < m; i++) cout << ans[i] << " ";
    cout << "\n";
}
```

Sample from the statement → `21 7 15 21 3`.

6. Common mistakes

- **Comparing parents instead of roots.** `p[a] == p[b]` is not a same-group test; use `find`.
- **Reading `sz` of a non-root.** Only `sz[find(x)]` is meaningful.
- **Linking non-roots.** `p[b] = a` without first replacing a and b by their roots silently corrupts the structure.
- **Forgetting the connectivity check** in MST problems: if fewer than $n - 1$ edges are taken (Kruskal) or fewer than n vertices are reached (Prim), print IMPOSSIBLE.
- **Overflow.** MST weights summed over $2 \cdot 10^5$ edges of weight 10^9 need `long long`; so does $s_1 \cdot s_2$ in pair counting.
- **Recursive find on degenerate trees** without union by size can overflow the stack; path halving avoids recursion entirely.
- **Prim with the Dijkstra key.** The priority must be the edge weight alone.
- **1-indexed vertices with `DSU d(n)`** — allocate n + 1.
- **Trying to delete from a DSU.** Reverse the process offline instead.

7. Practice problems

- CSES 1676 — Road Construction — https://cses.fi/problemset/task/1676
- CSES 1675 — Road Reparation — https://cses.fi/problemset/task/1675
- CSES 1677 — Network Breakdown (reverse time) — https://cses.fi/problemset/task/1677
- AtCoder Library Practice Contest A — Disjoint Set Union — https://atcoder.jp/contests/practice2/tasks/practice2_a
- Kattis — Union-Find — https://open.kattis.com/problems/unionfind
- Kattis — Minimum Spanning Tree — https://open.kattis.com/problems/minspantree
- Codeforces 25D — Roads not only in Berland — https://codeforces.com/problemset/problem/25/D
- Codeforces 1213G — Path Queries — https://codeforces.com/problemset/problem/1213/G
- Codeforces 1513D — GCD and MST — https://codeforces.com/problemset/problem/1513/D
- USACO 2016 US Open Gold — Closing the Farm — http://www.usaco.org/index.php?page=viewproblem2&cpid=646

8. Further reading

- Antti Laaksonen, *Competitive Programmer's Handbook*, Chapter 15 (Spanning trees), including the union-find structure.
- CP-Algorithms: "Disjoint Set Union" (https://cp-algorithms.com/data_structures/disjoint_set_union.html), "Minimum spanning tree — Kruskal with DSU" (https://cp-algorithms.com/graph/mst_kruskal_with_dsu.html), "Minimum spanning tree — Prim's algorithm" (https://cp-algorithms.com/graph/mst_prim.html).
- R. E. Tarjan, "Efficiency of a Good But Not Linear Set Union Algorithm", *Journal of the ACM*, 1975 — the inverse-Ackermann analysis.
- Looking ahead: Week 20 studies trees in depth (an MST is a tree, so everything there applies); Week 22 answers path-maximum queries on an MST online with binary lifting.